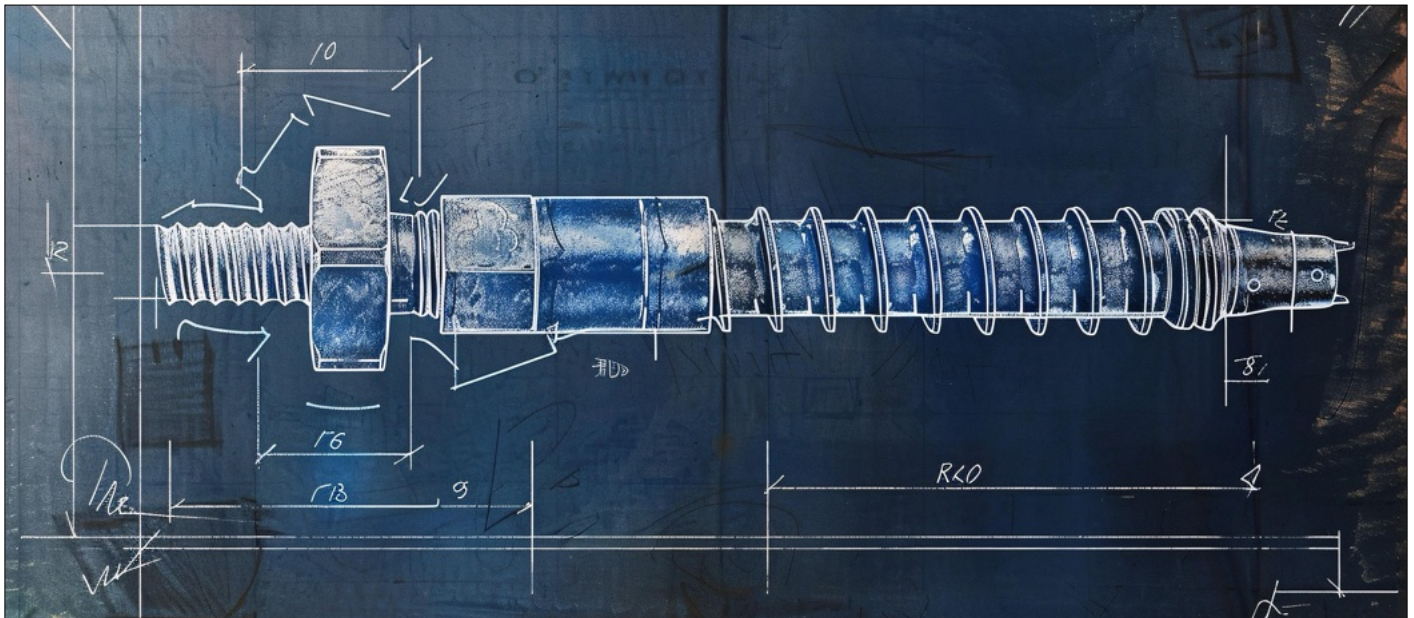




McMaster
University

ENGINEERING
Computing
& Software

Modelling Justifications to Support and Operationalize ML Pipelines

S. Mosser, K. Thuan-Phong Khuu
2026-10-04 — Part I
ACM/IEEE MODELS'26 (Málaga)



1

Understanding “Decision Drift”

2

jPipe & Justification Models

3

Modelling a Justification

4 Executing Justifications

Composing at Scale

6

5

Wrap up & Key Takeaways

Interactive parts



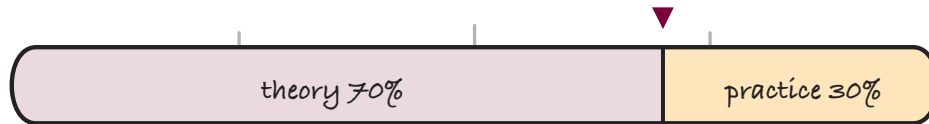
*We're brainstorming
as a group*



*You're working
on your laptop*

Overall organization

Part I



Part II



Before we start: Activate a Codespace



120 hours
included in
your free plan

You can also
install
jPipe locally

You'll need a
GitHub account

It can take
up to 5 minutes
to activate the
virtual environment.

<https://bit.ly/code-models26>

McMaster Centre for Software Certification

We *improve the practice* of **Software** Engineering and **Safety** Engineering

- **\$39M** of external funding
- **In-depth expertise** (since 2009)
 - **Automotive, Medical, Nuclear**
 - **IEEE TCSE Synergy Award 2024**
- **30** HQPs, **3** PDFs, **2** Research Engineers
- **Committed** to creating a ~~Brighter~~ World **Safer**



The engineer's reminder:
someone lives with your decisions.



Who are we?



Dr. Mark Lawford
Director



Dr. Sébastien Mosser
Associate Director



M. Alzaeim
Operations
Manager



Dr. R. Paige
Faculty
(CAS Chair)



Dr. A. Wassyng
Faculty



Dr. V. Pantelic
Senior Principal
Research Eng.



Dr. V. Bandur
Principal
Research Eng.



Dr. I. David
Faculty



What are we doing?

- **Leading research centre** in “**Safety-Critical Software-Intensive Systems**”
 - Examples: *Nuclear reactor shutdown systems, Vehicle over-the-air updates, ...*
 - Founder of the **Software Certification Consortium** in 2007 (23rd meeting)
- **Our approach**
 - **Do not invent your own problems!**
 - *Research intensively done with industrial partners (IEEE Synergy Award 2024)*
 - **Make it work!**
 - *Engineering tools and methods to solve real problems, at scale*

Examples of projects involving Requirements

- **Pacemaker (Boston Specific)**
 - Using **formal methods** to specify and build a pacemaker **conforms** to **textual requirements** (worldwide competition)
- **Insulin Pumps (FDA)**
 - **10-year consulting expertise** with the FDA to **review assurance cases** and assist them in **setting up guidelines** for the safety of insulin pumps
- **MRI-guided focused ultrasound machines (Arrayus)**
 - **Hazard analysis** & methodology for **assurance** (separating **technological** safety from **clinical** safety)
- **Phantom pain treatment (Axonic)**
 - Developing **methodology** and **justification models** to generate **certification paperwork**, ensuring the safety of the **implantable chips**.



Understanding Decision Drift

Do we really have a (requirements|models) problem?



Software Certification... in the age of AI

- Lots of research groups focus on the **certification** of **safety-critical systems** that include **AI components**
 - *Self-driving cars: 18-lane & 450k vehicle/day (401)*
 - *Object detection: Pedestrian or Scooter?*
- **AI has actually created different problems:**
 - Random dude **asks an LLM to write an app**
 - **The app is published** on the Apple/Play Store
 - **People get hurt**, or worse.



With GenAI & vibe-coding, **even a non-safety-critical app can hurt people**

Non-Safety-Critical: Mental Health

- **RELAI**: Respectful & Explainable AI (2019-2022)
 - *Suicidal risk evaluation for psychiatric ER care*
- **RÉSO-T**: Réseau, Écoute, Soutien, Oriente (2022-2025)
 - *Suicide prevention app in Abitibi-Temiscamingue*
- **FATES-MLOps**: Fairness, ... & MLOps (2024-2028)
 - *Operationalizing safety requirements in AI models*
- “Fun” fact:
 - In 2026, the integration of AI components is more regulated in **Banking** than in **Lethal Autonomous Weapon Systems (LAWS)** or **Healthcare / Wellbeing**. (Oopsie!)



Dr.G. Laperrière



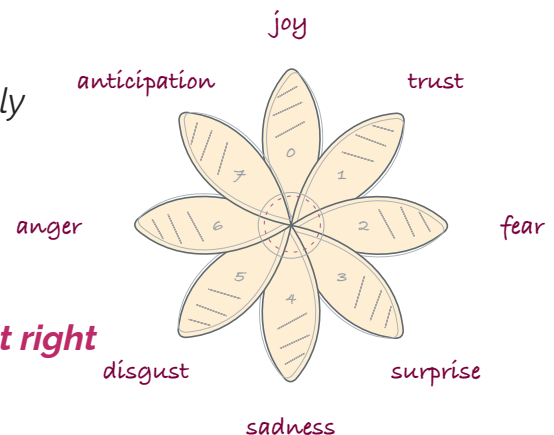
Dr. M.-J. Meurs

Catherine Lavoie



Tutorial: Zoom on Emotion Detection

- Emotion detection is **far from a trivial problem**
 - Requires **strong expertise in NLP** to do it correctly
 - But it's **easy to code** with nowadays libraries
- It's a **good example** to demonstrate how to:
 - Identify **requirements** & high-level **goals**
 - **Model the reasoning** justifying why we're **doing it right**
- **In a nutshell:**
 - **You receive “text”** as input, and **classify it** with its main “emotion”
 - For this tutorial, we're using **Plutchik's “wheel of emotions”**
 - Psychology research: eight major emotions and a proximity rule



Let's ask Claude to write it for me!

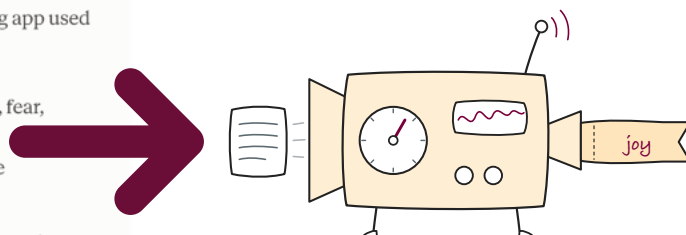


Prompt

You are an ML engineer. Build an emotion detection classifier for a journaling app used by teenagers (13-18) to support their mental health.

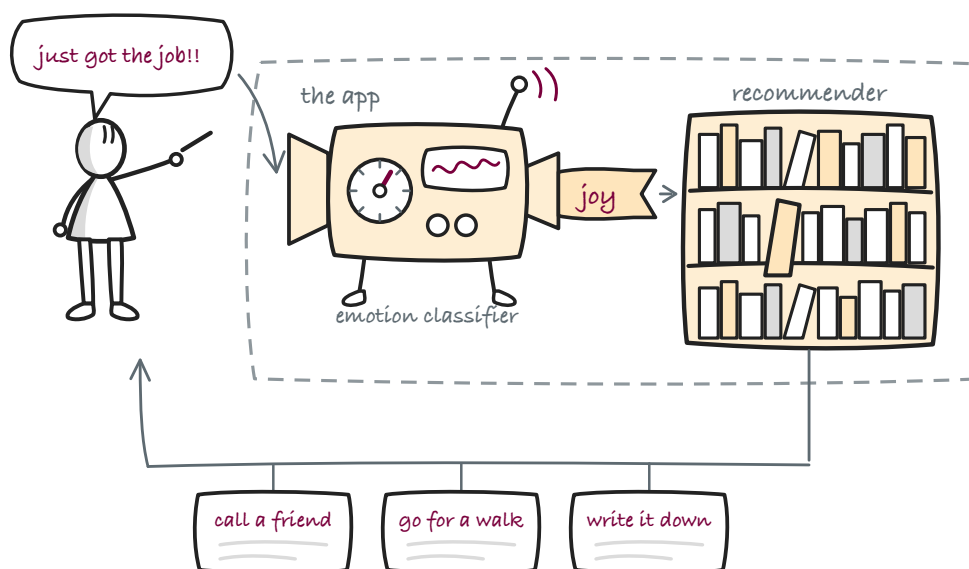
- Input: free-text journal entries (50-500 words)
- Output: top 3 emotions with confidence scores, from: joy, sadness, anger, fear, anxiety, loneliness, hope, shame
- Constraints: runs on-device, <100ms inference, no data leaves the phone
- Flag entries indicating crisis risk for in-app support resources

Recommend a model architecture, training dataset strategy, and evaluation metrics. Note any bias or safety risks specific to this age group.



Who needs **engineers** when an LLM can **vibecode it** in 5 minutes?

A few (vibecoded) iterations later



Which classifier would you ship in the app?



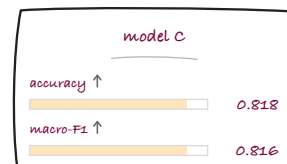
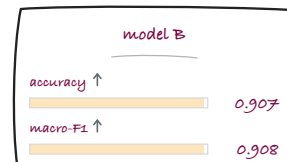
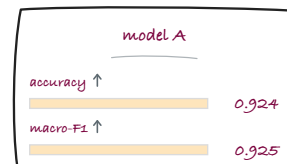
• Metrics measured:

• Accuracy

- Percentage of predictions the model got right

• Macro-F₁

- Average success while minimizing (1) missing targets and (2) false alarms



↑ = the direction that is better

Which of these three “vibe-trained” models should we deploy?



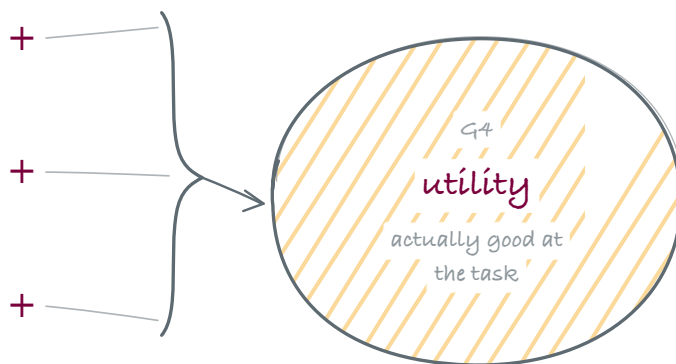
We're dealing with implicit requirements



R1 • single label per record
exactly one emotion, no multi-label

R2 • eight Plutchik labels
the eight primaries, nothing else

R17 • report accuracy & macro-F₁
the headline performance numbers

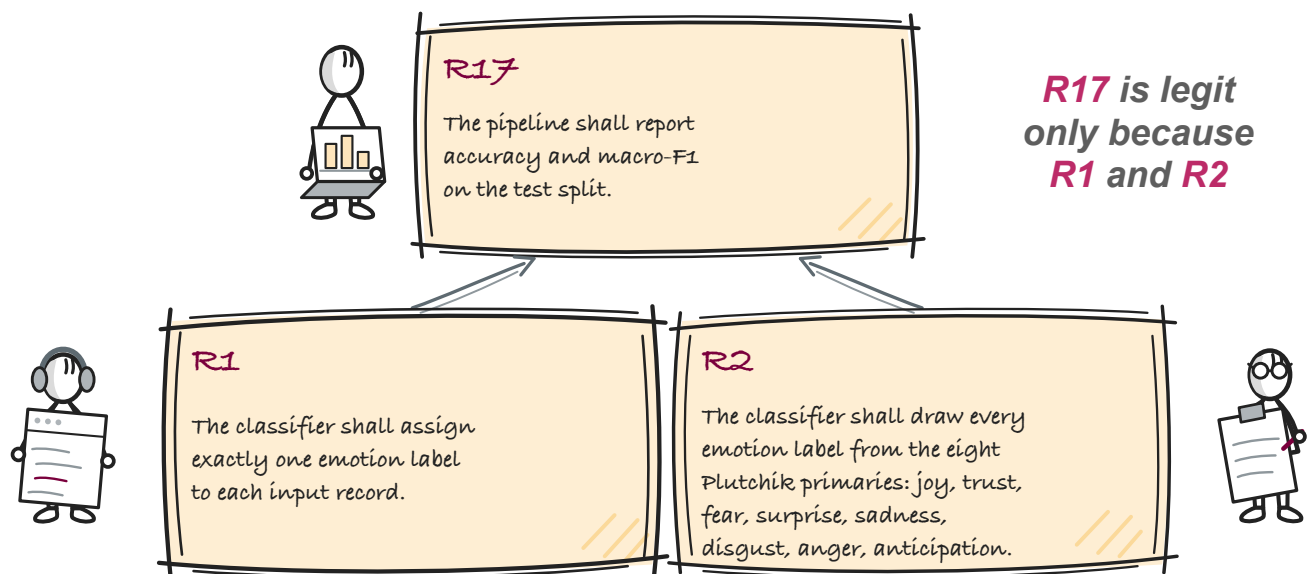


The **design choices** made in the training pipeline are driven by these **requirements** to address the **predictive performance** of the classifier

They are very classical, so the LLM took them for granted!



High-level requirements & Dependencies



Wait! Have we looked at the dataset?

protected attributes

text	gender	ethnicity	age_group	emotion
They kept their eyes squeezed shut...	female	east_asian	adult	fear
The heart hammered against the ribs...	female	arabic	young	fear
A sour, heavy odor filled the room...	male	arabic	adult	disgust
I pace the living room floor until...	nonbinary	arabic	adult	anticipation
They huddled together in the hallway...	female	east_asian	adult	fear

model A	
accuracy ↑	0.924
macro-F1 ↑	0.925

So we might be learning more from {gender, ethnicity, age_group} than from {text}

The Fairness goal is supported by four reqs

R22 · 27-cell counterfactual grid
3 genders × 3 ethnicities × 3 ages

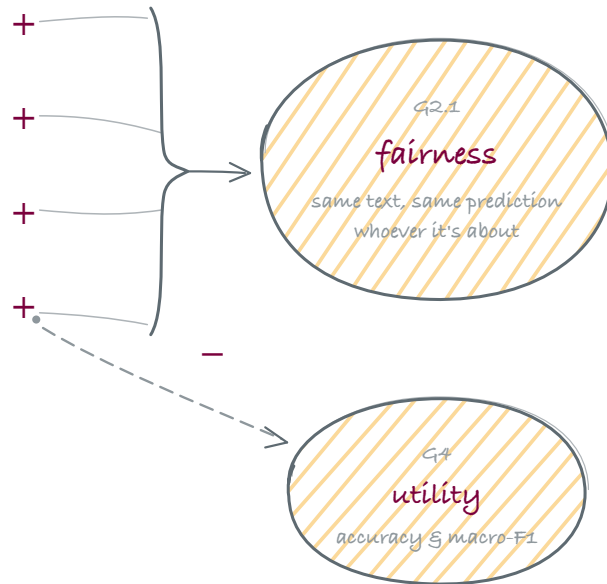
R20 · same emotion per CF group
all 27 variants get one answer

R18 · report flip-rate
residual sensitivity, measured

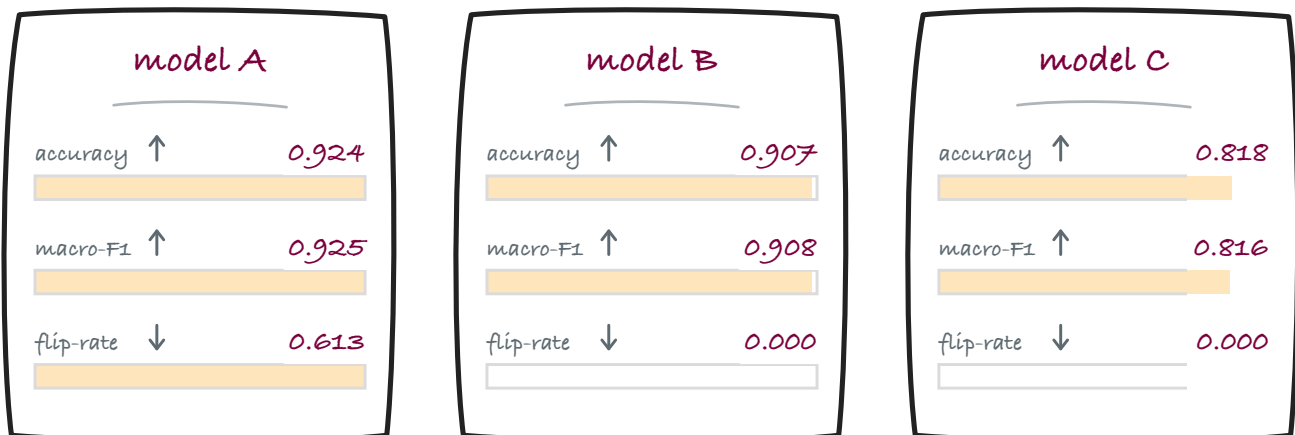
R13 · remove protected attributes
the model cannot see identity

+ · helps the goal — · hurts it

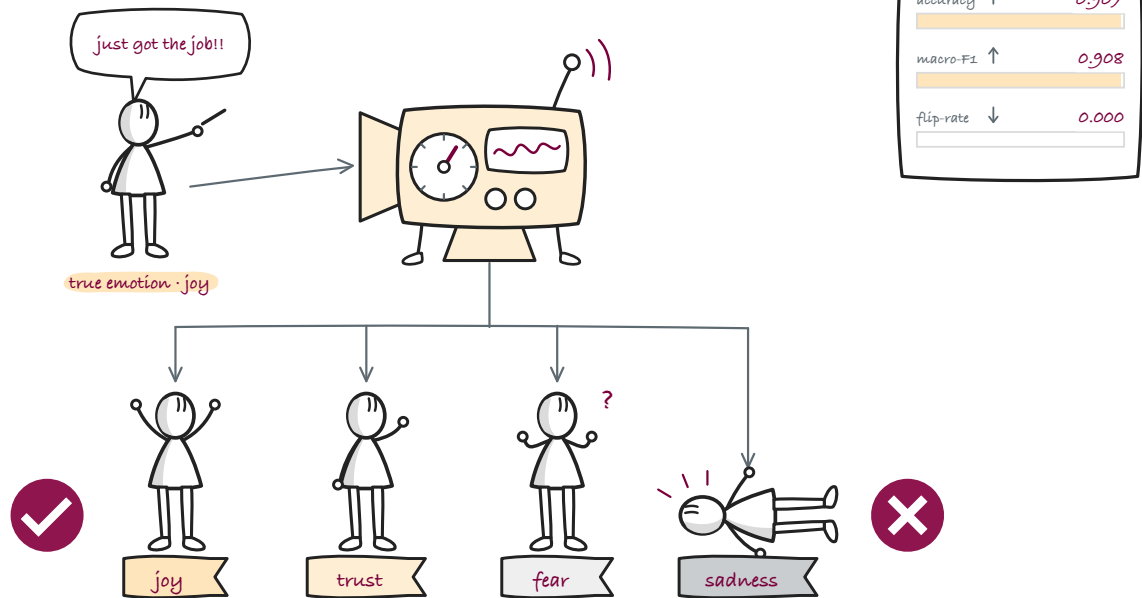
R13 buys fairness with accuracy —
the trade v2 and v3 both make



Let's consider the counterfactual flip-rate

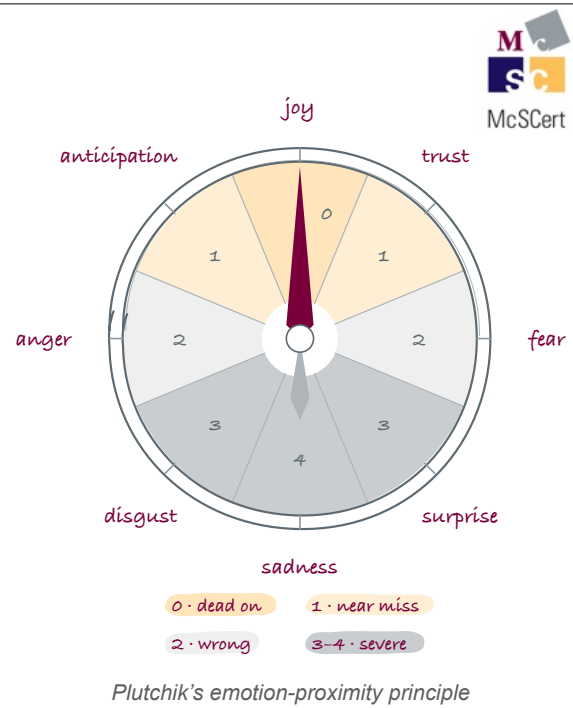


Wait! Not all errors are created equal



Let's zoom on Safety

- **Safety:**
 - “The condition of **being safe** from undergoing or **causing harm, injury, or loss**”
- **Problem:**
 - We're **recommending activities** based on the **detected emotion**.
 - We **reinforce** the “good” ones, and **deflect** the “bad” ones.
 - **Off-by-one errors are fine**, but more is a **safety hazard** for the end user.



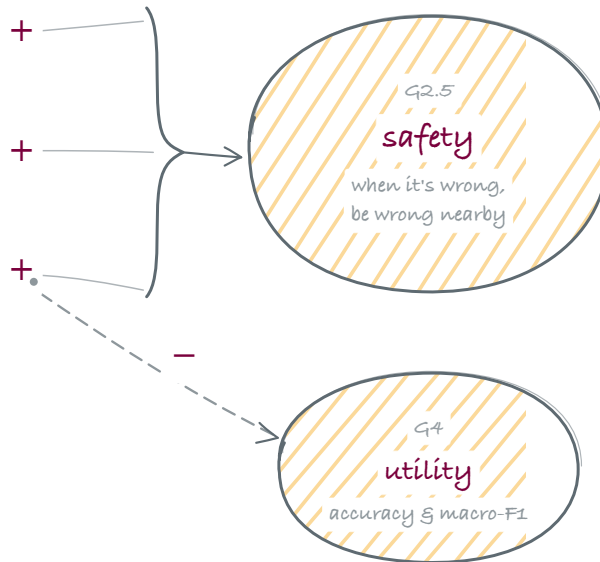
A “safer” detection will hurt “performance”

R15 · wheel distance metric
 $\min(|i-j|, 8 - |i-j|)$ on the wheel

R19 · distance & severe-error rate
 how bad, and how often it's bad

R14 · prefer near-miss errors
 safer failures — and fewer right answers

+ · helps the goal — · hurts it
 R14 buys safer failures with accuracy —
 the trade v3 makes and v2 does not



Impact on Measurements

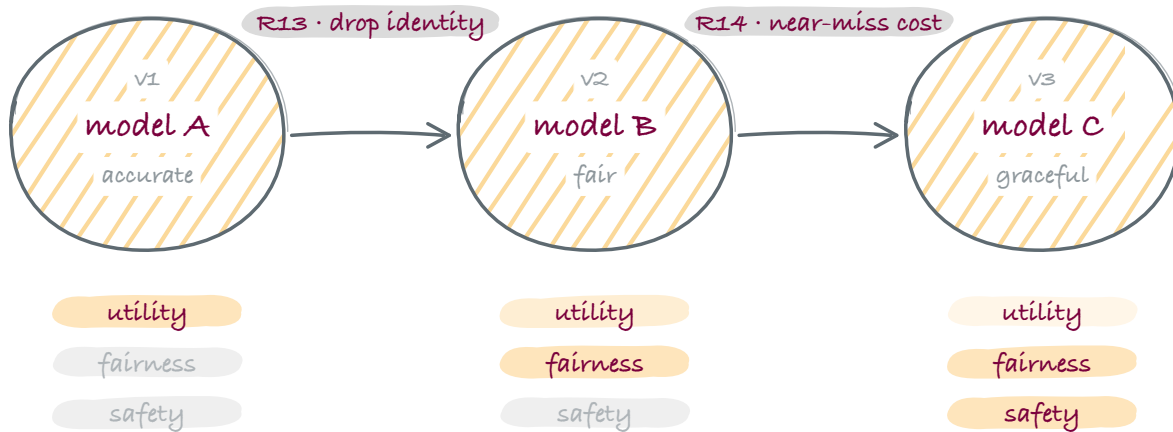
Performance

Fairness

Safety

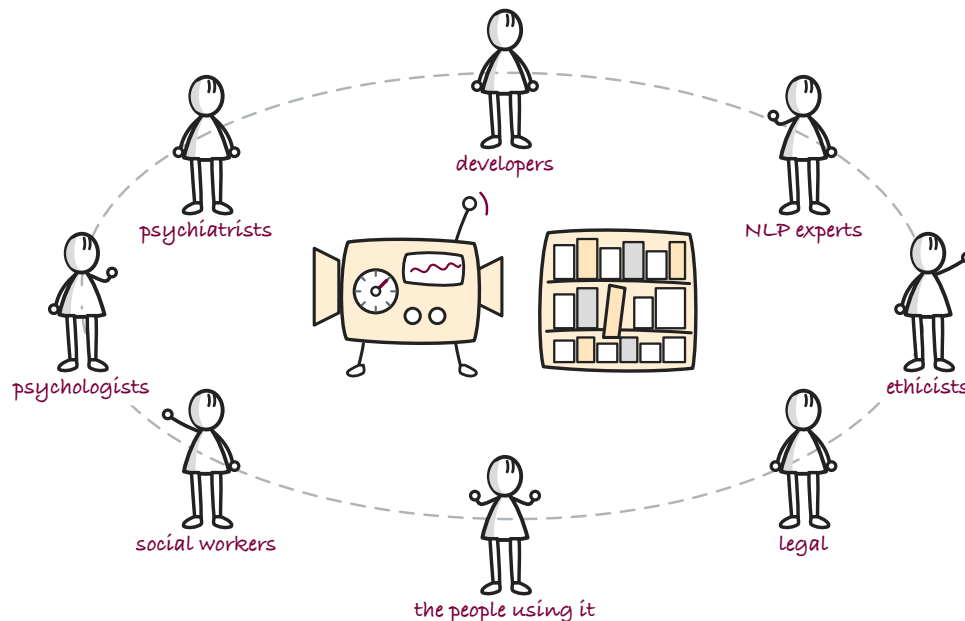


So..., long story short...

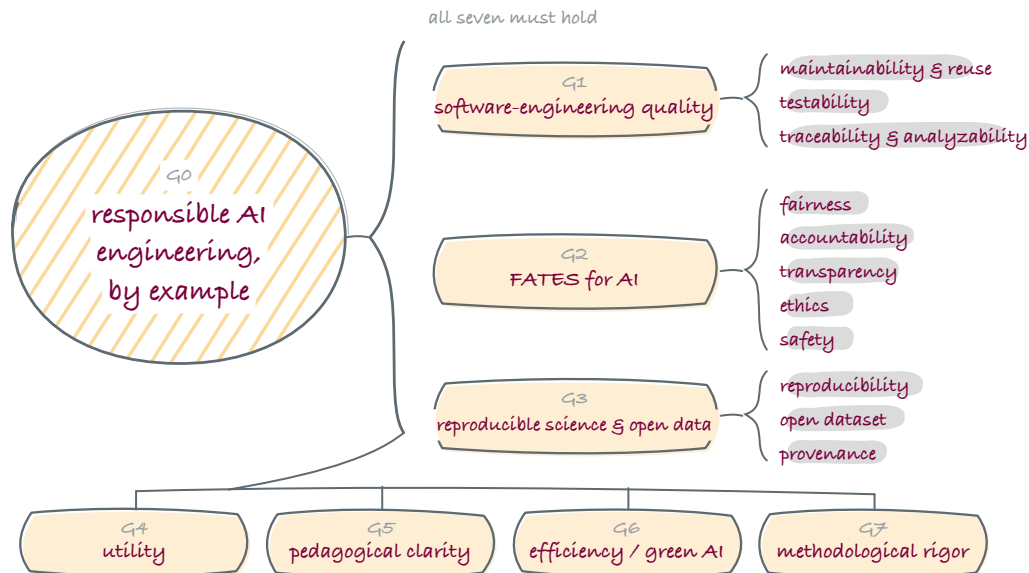


And this is just a **very simplified version** of the real story

Real Story: Stakeholders Elicitation



(Soft) Goals Decomposition



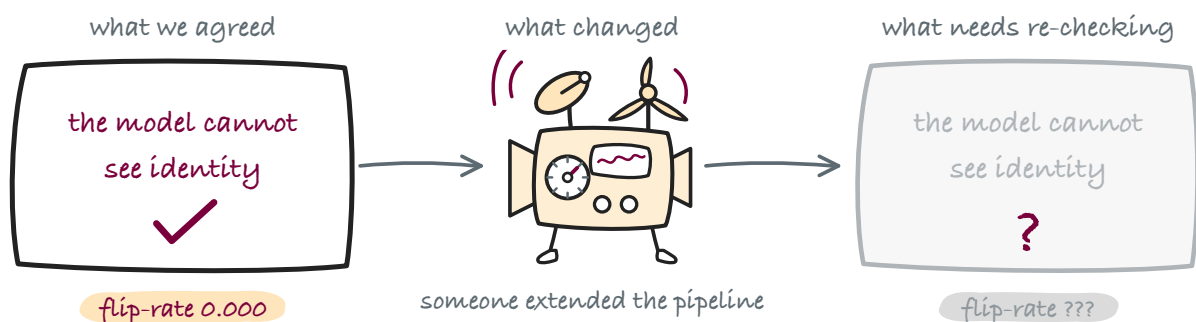
From “5-minute vibecoding” to “doing it right”



- | | | |
|-----------------------------|--------------------------------|-----------------------------|
| R1 · one label per record | R13 · drop protected attrs | R25 · frozen dataset |
| R2 · eight Plutchik labels | R14 · prefer near-miss errors | R26 · rich-tier, balanced |
| R3 · dataset column schema | R15 · wheel distance metric | R27 · per-step traceability |
| R4 · no empty columns | R16 · train & print metrics | R28 · legible stages |
| R5 · allowed values only | R17 · accuracy & macro-F1 | R29 · one implementation |
| R6 · group_id invariant | R18 · counterfactual flip-rate | R30 · fixed seeds |
| R7 · interpretable features | R19 · distance & severe rate | R31 · self-documenting code |
| R8 · linear classifier | R20 · same emotion per group | R32 · core logic tested |
| R9 · CPU-only, no network | R21 · disjoint splits | R33 · persist model |
| R10 · pinned dependencies | R22 · 27-cell CF group | R34 · persist metrics |
| R11 · exact env from lock | R23 · committed generator | R35 · classify tool |
| R12 · keep protected attrs | R24 · inject identity bias | |

The image shows a web browser displaying the GitHub repository page for 'jpipe-case-emotions'. The main content area shows the README file, which is titled 'Emotion Detection: One Classifier, Three Versions'. The README text describes the project's goal of building a model that is accurate, fair, and graceful in its failure. It lists two authors: Sébastien Mosser and Kalvin Thuan-Phong Khuu. The 'Overview' section explains that the model is built using the same emotion classifier three times over the same committed data, each version adding one quality the previous one lacked. It also mentions that the task is deliberately classic - single-label classification of short free-text messages into the eight primary emotions of Plutchik's wheel. The 'Source code available on GitHub' note is prominently displayed in red. The left sidebar shows the repository's file structure, including 'src', 'tests', and 'README.md'.

From Requirements to Decisions to... Drift!



"Duh, bro, just write some unit tests in python, you noob"

Sure, it's “technically possible”...



```
@pytest.mark.parametrize("tier", TIERS)
def test_counterfactual_group_is_the_full_27_cell_identity_grid(tier: str) -> None:
    frame = load_split(tier, "counterfactual").frame
    axes = list(PROTECTED_ATTRIBUTES)
    full_grid = set(itertools.product(*PROTECTED_ATTRIBUTES.values()))
    assert len(full_grid) == _CF_GROUP_SIZE
    for group_id, group in frame.groupby("group_id"):
        assert len(group) == _CF_GROUP_SIZE, f"{tier}: group {group_id} has {len(group)} rows"
        combos = set(map(tuple, group[axes].itertuples(index=False, name=None)))
        assert combos == full_grid, f"{tier}: group {group_id} is not the full identity grid"

def test_flip_rate_as_dict_flat_keys() -> None:
    score = FlipRateScore(overall=0.03, per_axis={"gender": 0.02, "ethnicity": 0.04, "age_group": 0.02})
    assert score.as_dict() == {
        "flip_rate_overall": 0.03,
        "flip_rate_gender": 0.02,
        "flip_rate_ethnicity": 0.04,
        "flip_rate_age_group": 0.02,
    }

@pytest.mark.parametrize("cost_sensitive", [False, True], ids=["v2", "v3"])
def test_fair_model_never_flips_within_a_counterfactual_group(cost_sensitive: bool) -> None:
    train = load_split("smoke", "train").drop(*PROTECTED_ATTRIBUTES)
    counterfactual = load_split("smoke", "counterfactual")
    cf_features = counterfactual.drop(*PROTECTED_ATTRIBUTES)
    classifier = EmotionClassifier(cost_sensitive=cost_sensitive).train(train)
    score = flip_rate(counterfactual.frame, classifier.predict(cf_features).tolist())
    assert score.overall == 0.0
    assert all(rate == 0.0 for rate in score.per_axis.values())

def test_flip_rate_fair_model_scores_zero() -> None:
    frame = pd.concat([_cf_group(0), _cf_group(1)], ignore_index=True)
    predictions = ["joy"] * 27 + ["anger"] * 27
    score = flip_rate(frame, predictions)
    assert score.overall == pytest.approx(0.0)
    assert all(rate == pytest.approx(0.0) for rate in score.per_axis.values())

def test_flip_rate_overall_counts_flipped_groups() -> None:
    frame = pd.concat([_cf_group(0), _cf_group(1)], ignore_index=True)
    group0 = ["joy"] * 26 + ["sadness"]
    group1 = ["anger"] * 27
    score = flip_rate(frame, group0 + group1)
    assert score.overall == pytest.approx(0.5)

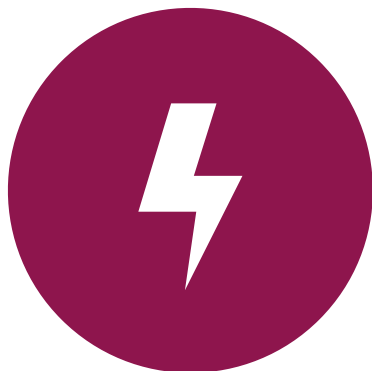
def test_flip_rate_format_report_shape() -> None:
    report = FlipRateScore(
        overall=0.03, per_axis={"gender": 0.02, "ethnicity": 0.04, "age_group": 0.02}
    ).format_report()
    assert "flip-rate (overall):" in report
    for axis in PROTECTED_ATTRIBUTES:
        assert f"{axis}:" in report

def test_flip_rate_per_axis_isolates_the_varying_axis() -> None:
    frame = _cf_group(0)
    by_gender = {"female": "joy", "male": "anger", "nonbinary": "fear"}
    predictions = [by_gender[g] for g in frame["gender"]]
    score = flip_rate(frame, predictions)
    assert score.overall == pytest.approx(1.0)
    assert score.per_axis["gender"] == pytest.approx(1.0)
    assert score.per_axis["ethnicity"] == pytest.approx(0.0)
    assert score.per_axis["age_group"] == pytest.approx(0.0)
```

But tracing back this to our high-level goal will be challenging



So, let's take a step back!

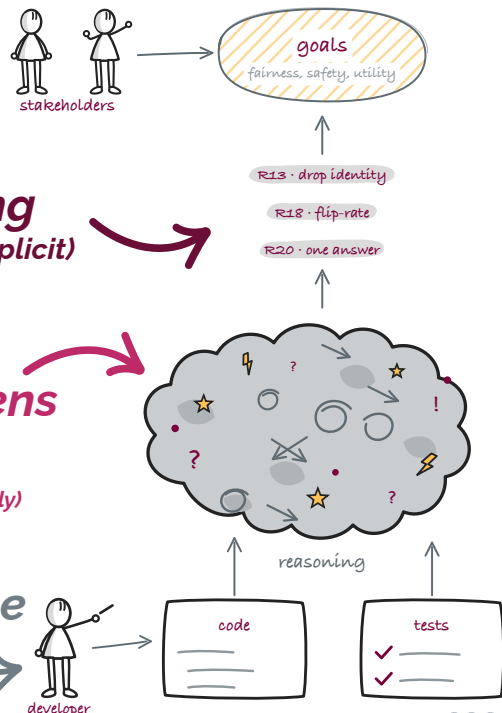


Existing
(but often implicit)

Magic happens

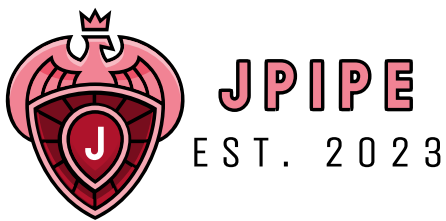
(and troubles)
(mostly troubles, actually)
(Like BIG troubles)

State of practice

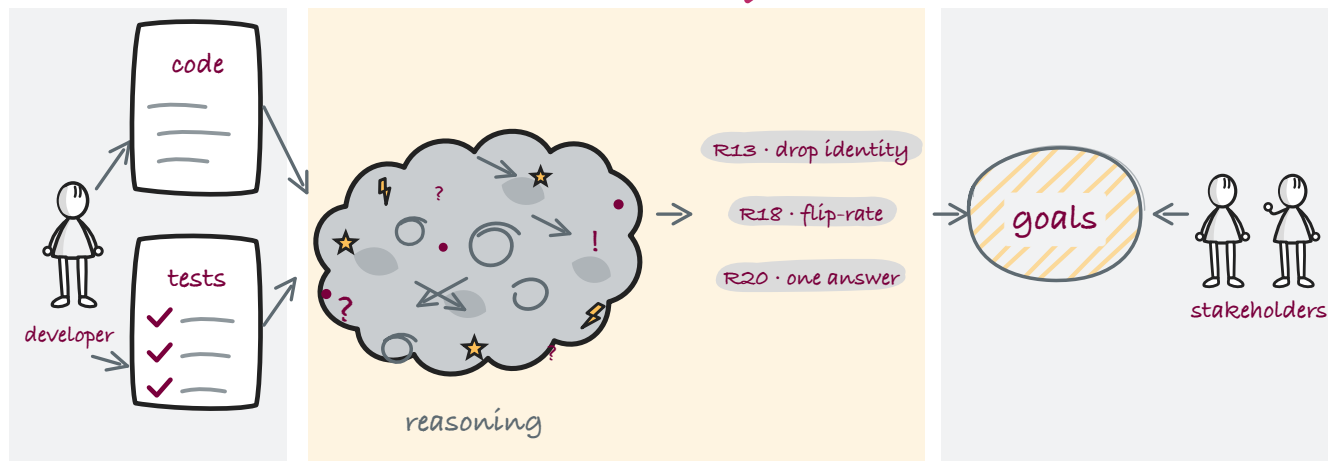


jPipe & Justification Models

Don't worry, we built a tool to help you!



Without any surprise...



We align with developers' practices (Python code, tests)

We offer **jPipe**, a way to **model the justification, operationalize it, and integrate it** to avoid decision drift

We work with **what we have** (and incrementally add what we'll discover later)

"To explain" is not "To justify"

explain verb

Save Word

ex·plain | \ ɪkˈsplān \

explained; explaining; explains

Documentation

Essential Meaning of *explain*

- 1 : to make (something) clear or easy to understand
// I need a lawyer to *explain* this contract to me.
// The professor *explained* the poem to the class.
See More Examples
- 2 : to tell, show, or be the reason for or cause of something
// Scientists could not *explain* the strange lights in the sky.
// I don't know how to *explain* the dog's strange behavior.
See More Examples

justify verb

Save Word

jus·ti·fy | \ ˈjə-stə-,fi \

justified; justifying

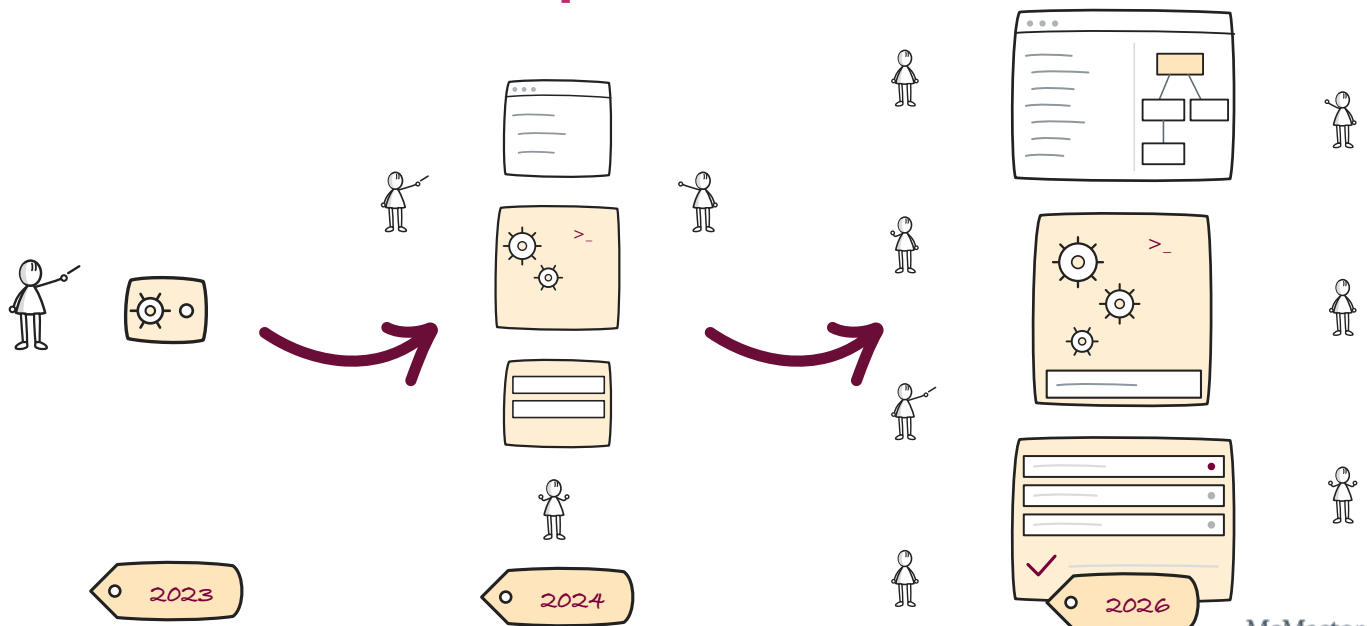
Actionable requirements validation

Essential Meaning of *justify*

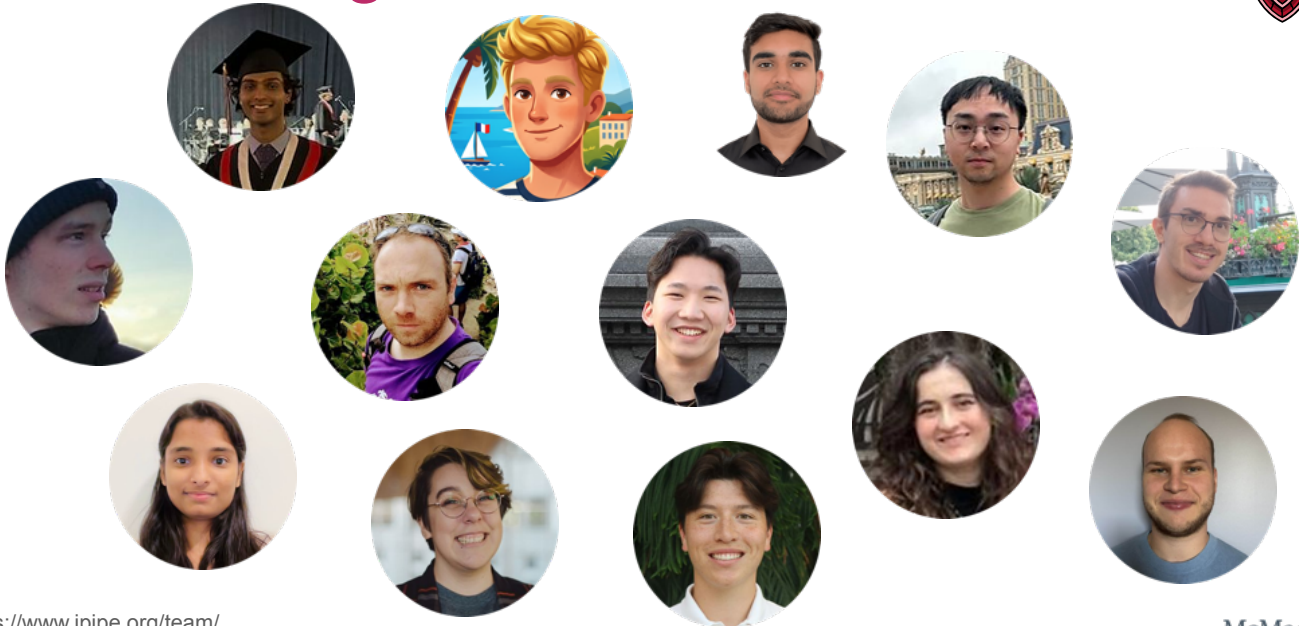
- 1 : to provide or be a good reason for (something) : to prove or show (something) to be just, right, or reasonable
// He tried to *justify* his behavior by saying that he was being pressured unfairly by his boss.
// The fact that we are at war does not *justify* treating innocent people as criminals.
// It's hard to *justify* the cost of a new car right now. = It's hard to *justify* spending money on a new car right now.
- 2 : to provide a good reason for the actions of (someone)
// Why should I have to *justify myself* [=to provide an explanation for my actions] when it was their fault?

We're defending that the system is **Just, Right, and Reasonable.**

From Proof of concept to Research tool

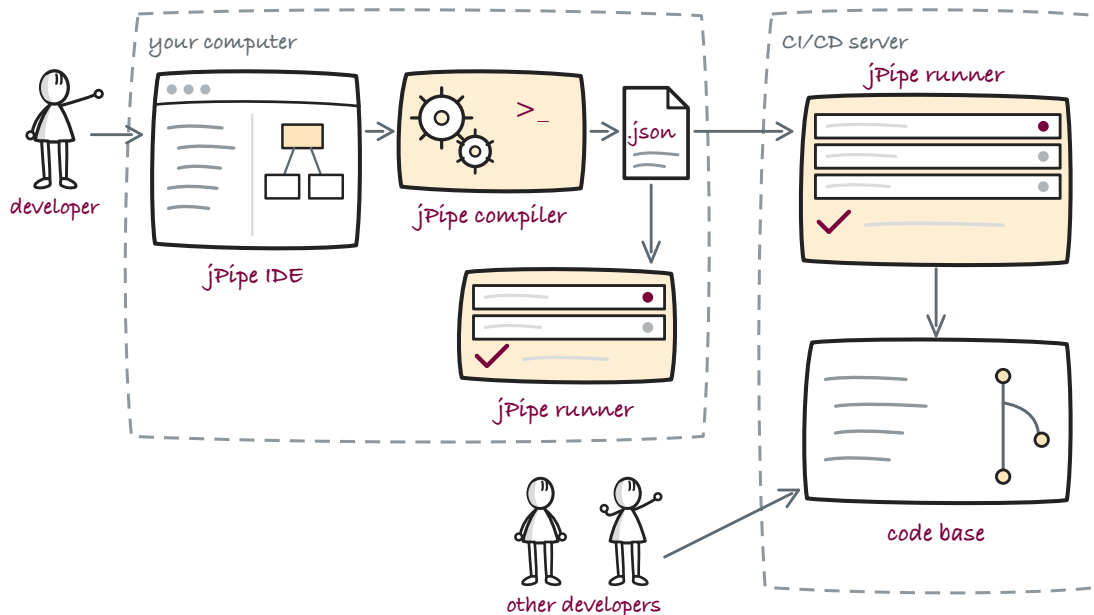


It takes a village to build a tool!



<https://www.jpipe.org/team/>

A Tool? How does it work?

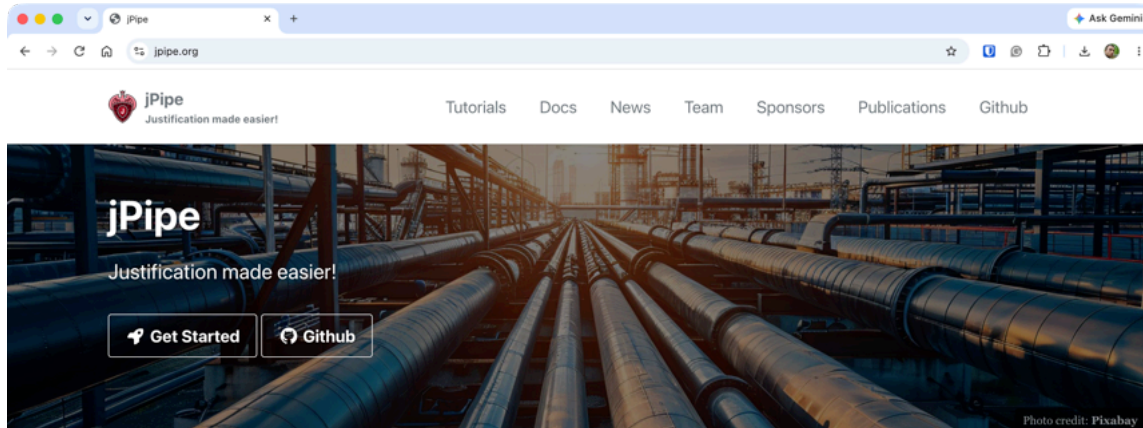


jPipe is an Open Source Ecosystem



Project						
jpipe-compiler	build passing	quality gate passed	coverage 91%	commits 605	release v2.5.0	Stars 30
jpipe-vscode	build passing	quality gate passed	coverage 84%	commits 832	release v1.8.1	Stars 6
jpipe-runner	build passing	quality gate passed	N/A	commits 180	release v3.5.3	Stars 7

Open Source (MIT), free to use (*open to consulting*), state-of-the-art development



The **jPipe environment** supports the definition of justification to support software maintenance activities, bringing **lightweight safety modelling** features to software developers. The name comes from **justified pipelines**, as the key idea is to design an environment supporting the justification of C/CD pipelines by design.

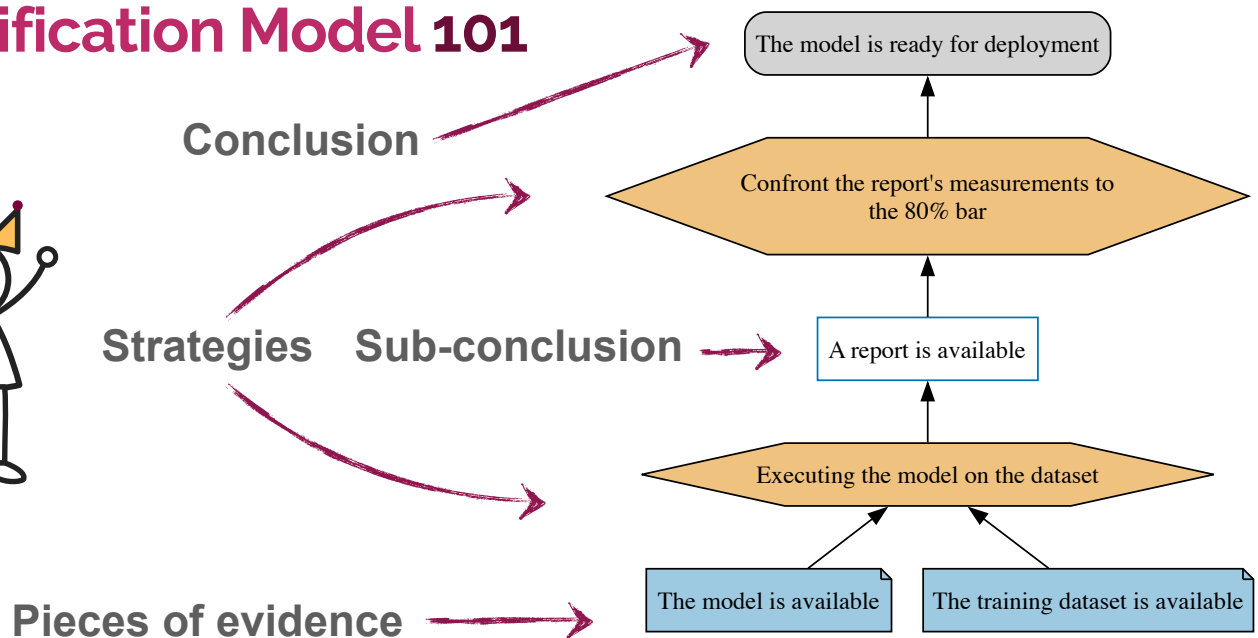
jPipe is designed at [McSCert](#) in Hamilton, Ontario ([McMaster University](#)).

My first justification

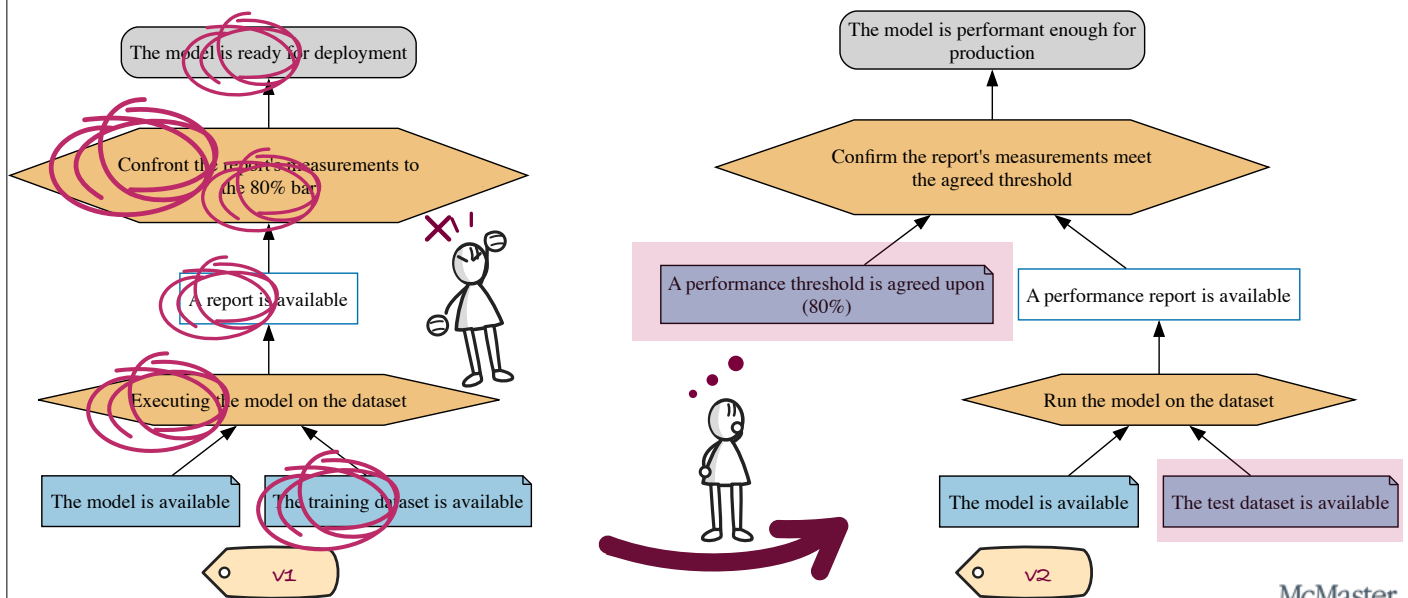


- You've just trained your pipeline
 - For now, only **F1-score** and **accuracy** matter
 - You have a **threshold of 80%** for both
- What is your reasoning for claiming that you can deploy your model, assuming these two thresholds are met?
 - Which **pieces of evidence** do you need?
 - How do they contribute to "**I can deploy**"?

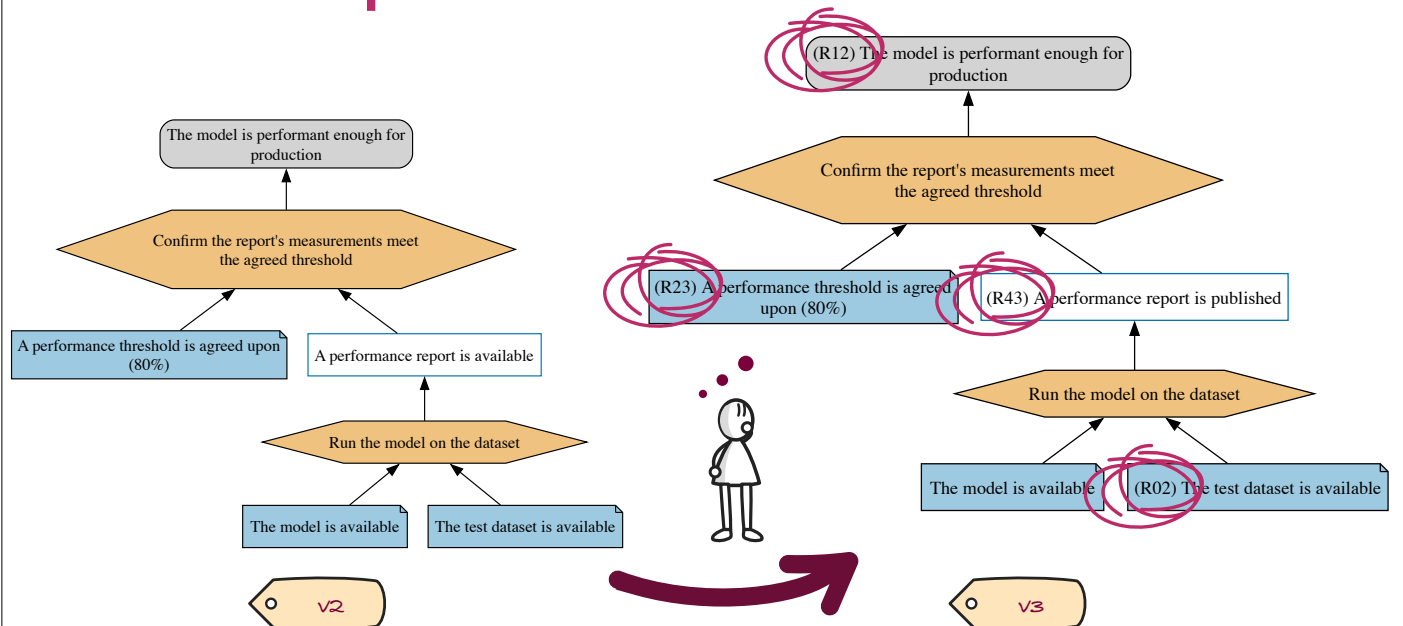
Justification Model 101



Incrementally refining the model



Link with requirements?



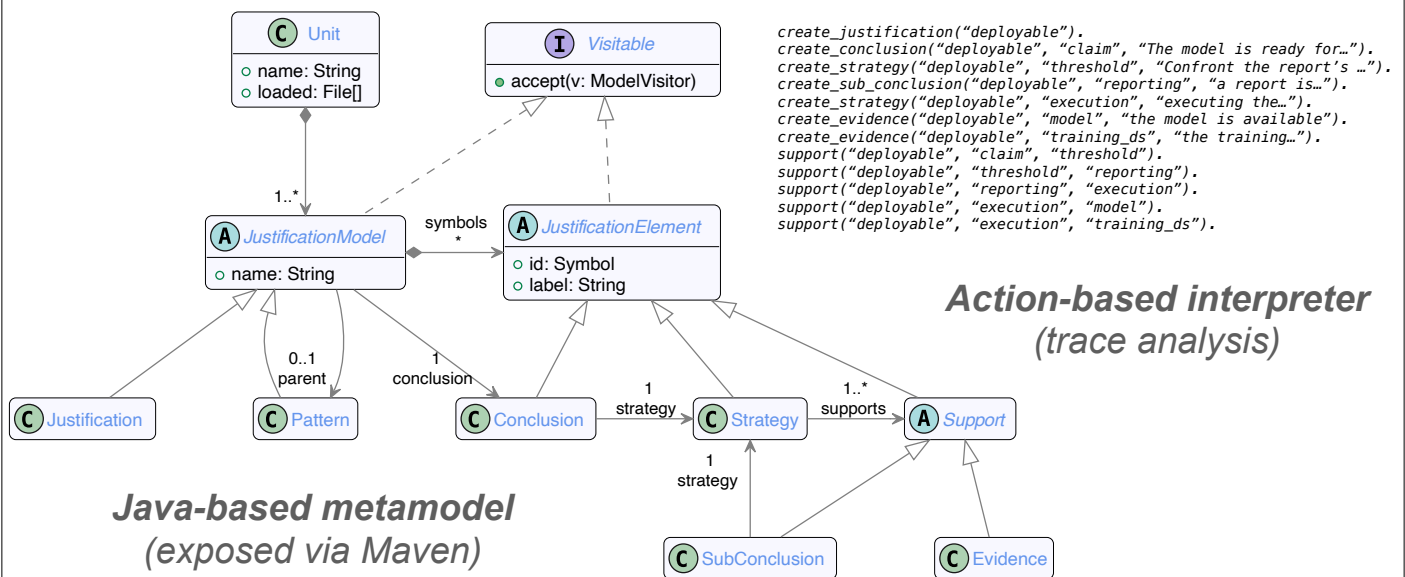
A domain-specific language approach

```
justification deployable {
    conclusion      claim      is "The model is ready for deployment"
    strategy        threshold  is "Confront the report's measurements to the 80% bar"
    sub-conclusion  reporting   is "A report is available"
    strategy        execution   is "Executing the model on the dataset"
    evidence        model       is "The model is available"
    evidence        training_ds is "The training dataset is available"

    threshold supports claim
    reporting  supports threshold
    execution  supports reporting
    model      supports execution
    training_ds supports execution
}
```

jPipe models are text, and can be shared, versioned and exchanged as such.

The language maps to a meta-model

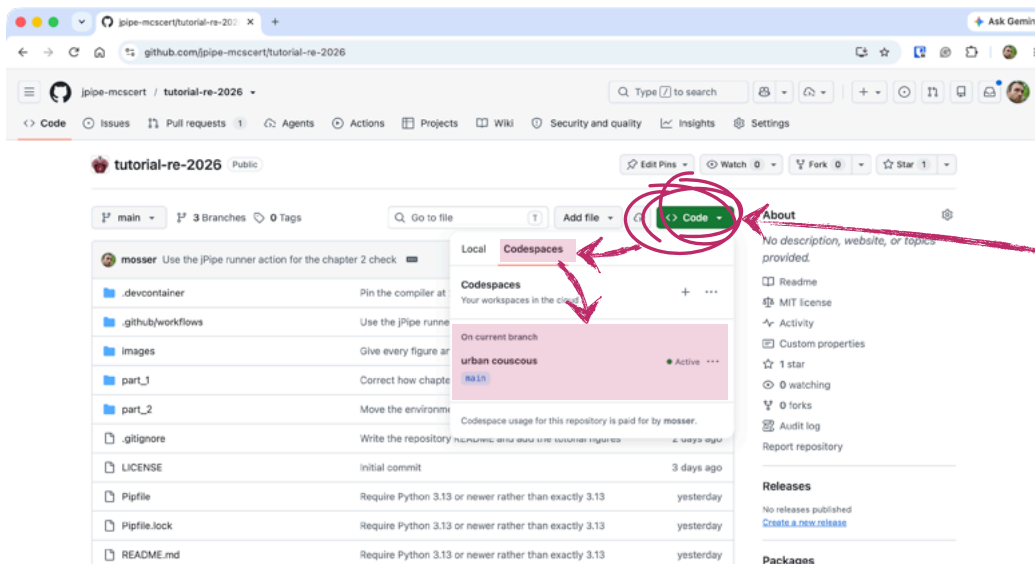


Let's open jPipe IDE on your laptop



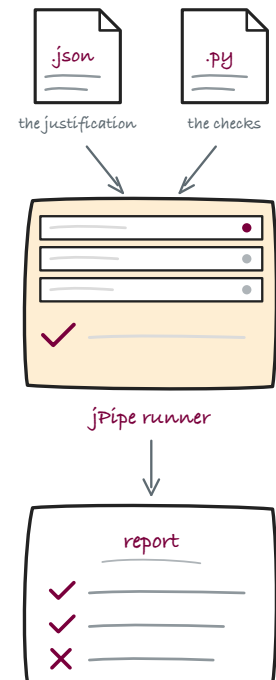
- Open your **Codespace** or local IDE
 - If not already: <https://bit.ly/code-models26>
- Navigate to **part_1 >> chapter_2 >> deployable.jd**
- **Tasks**
 1. **Navigate** the model and its preview
 2. Play with the **model editing tools**
 - Completion, refactor, auto-format, ...
 3. Activate the **Diagnostic mode**.

Finding back your Codespace



<https://bit.ly/jpipe-models26>

- **Constraints** coming from **ML stakeholders**
 - Use what **ML developers usually use** (e.g., Python)
 - Make it **look like what they already know** (e.g., unit tests)
 - Make its **usage as seamless as possible** (e.g., 1-line invocation)
- **The execution engine takes two inputs**
 - **A justification**, exported as JSON
 - **Some Python code** to implement the validation checks
 - Executing the **justification produces a report**

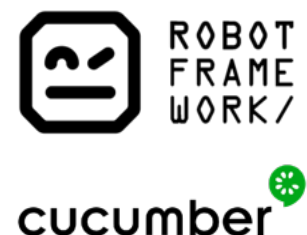


```
mosser@belphegor examples % jpipe-runner --diagram deployable -l deployable.py deployable.json
```

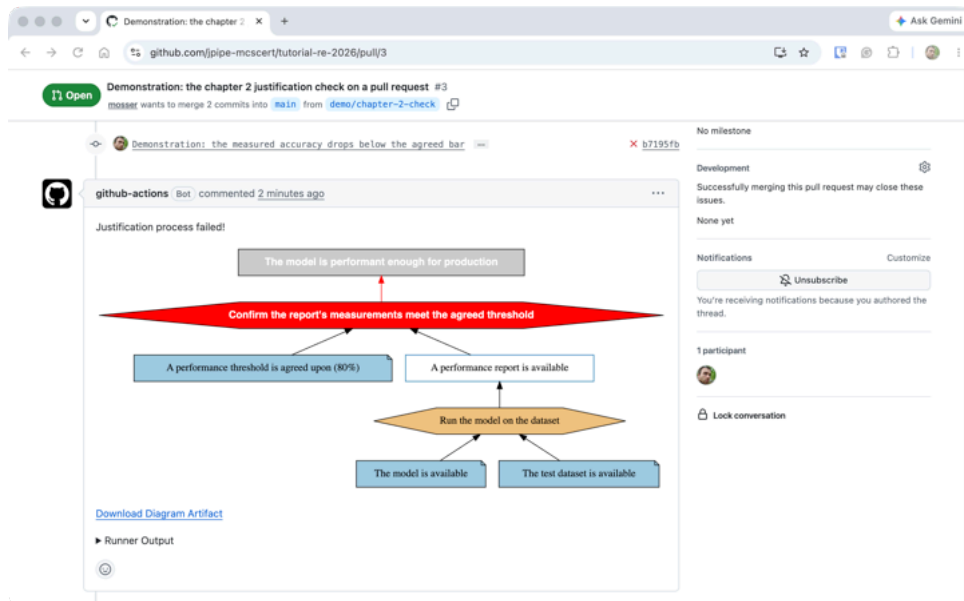
THE UNIVERSITY OF CHICAGO

```
jPipe Files
=====
jPipe Files.Justification :: deployable
=====
evidence<deployable:model> :: The model is available | PASS |
evidence<deployable:training_ds> :: The training dataset is available | PASS |
strategy<deployable:execution> :: Executing the model on the dataset | PASS |
sub-conclusion<deployable:reporting> :: A report is available | PASS |
strategy<deployable:threshold> :: Confront the report's measuremen... | PASS |
conclusion<deployable:claim> :: The model is ready for deployment | PASS |
=====
jPipe Files
1 justification, 6 passed, 0 failed, 0 skipped
```

(Inspired by)



Integration with CI/CD pipelines

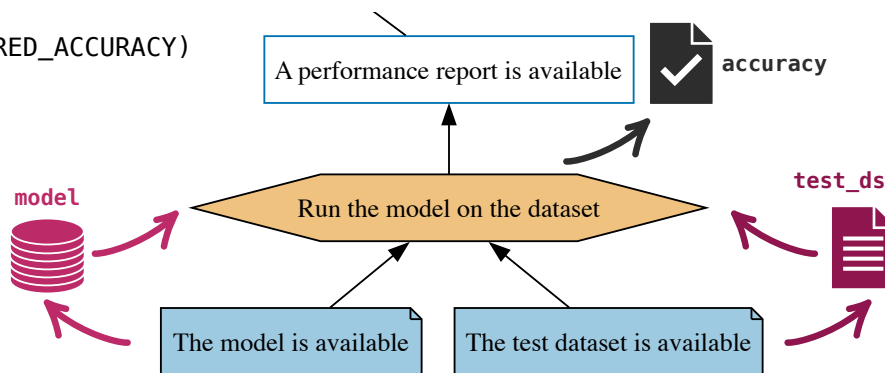


How to implement an element?

Elements **produce** and/or **consume** artifacts

```
@jpipe_link("deployable:execution")
@jpipe(produce=["accuracy"], consume=["model", "test_ds"])
def run_the_model_on_the_dataset(model, test_ds,
                                produce: JpipeProduce) -> bool:
    """[strategy] Run the model on the dataset"""
    # Real run code goes here.
    if not (model and test_ds):
        return False
    produce("accuracy", MEASURED_ACCURACY)
    return True
```

Validation rules that produced elements are **consumed** and **accessed** downstream

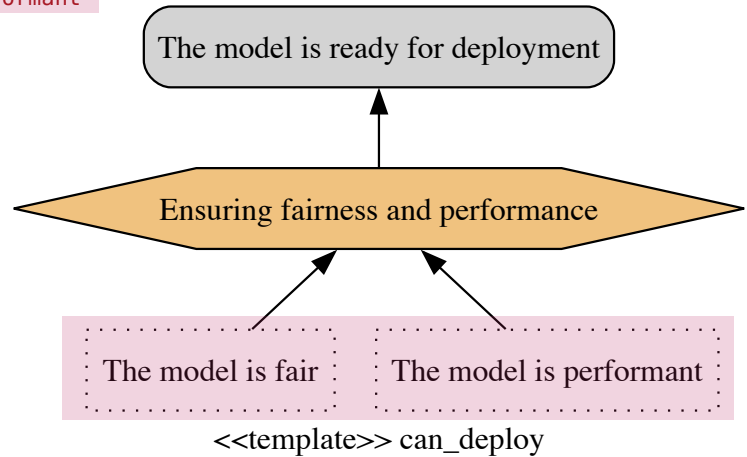


Templates to enforce a top-down structure

```
template can_deploy {
  conclusion claim is "The model is ready for deployment"
  strategy altogether is "Ensuring fairness and performance"
  @support fair is "The model is fair"
  @support performant is "The model is performant"

  altogether supports claim
  fair supports altogether
  performant supports altogether
}
```

Templates declare **abstract support** elements (**@support**) that will be replaced by **pieces of evidence** or **sub-conclusions** when instantiating them



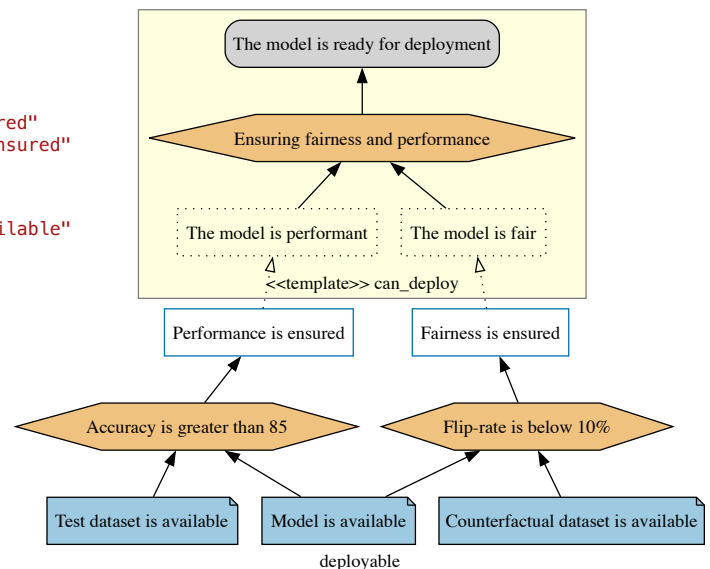
Example of Template Implementation

```
justification deployable implements can_deploy {
  // Hooks from template
  sub-conclusion can_deploy:fair is "Fairness is ensured"
  sub-conclusion can_deploy:performant is "Performance is ensured"

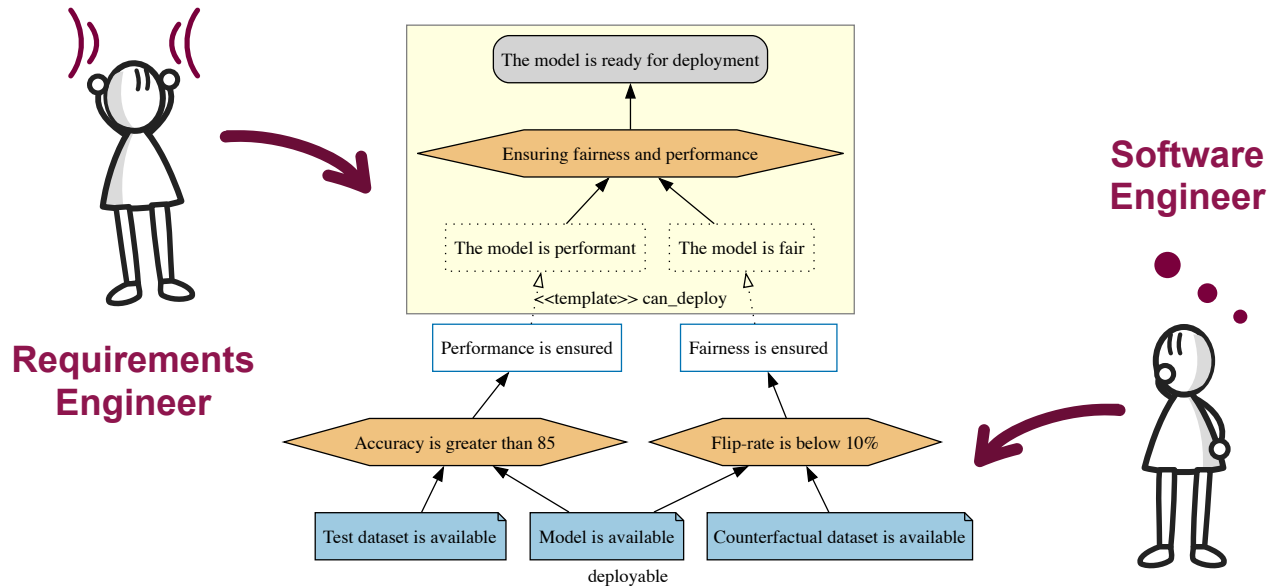
  // Fairness branch
  strategy flip is "Flip-rate is below 10%"
  evidence counterfactual is "Counterfactual dataset is available"
  flip supports can_deploy:fair
  model supports flip
  counterfactual supports flip

  // Performance branch
  strategy accuracy is "Accuracy is greater than 85%"
  evidence test is "Test dataset is available"
  accuracy supports can_deploy:performant
  model supports accuracy
  test supports accuracy

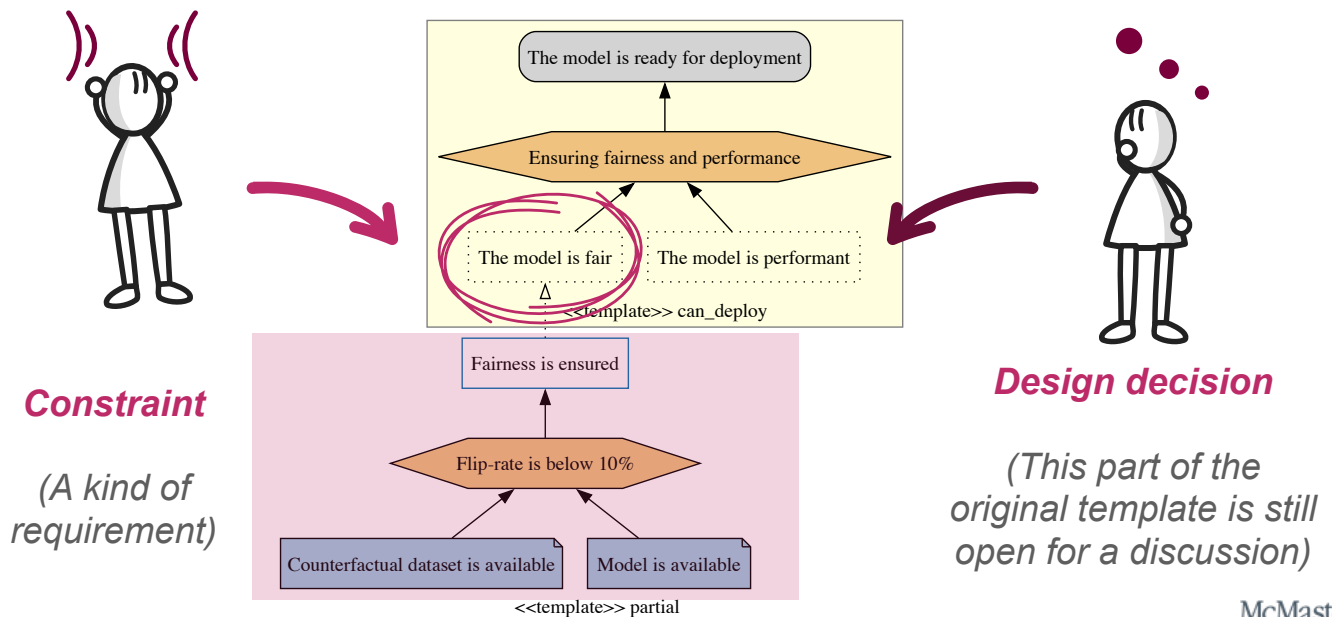
  // Shared elements
  evidence model is "Model is available"
}
```



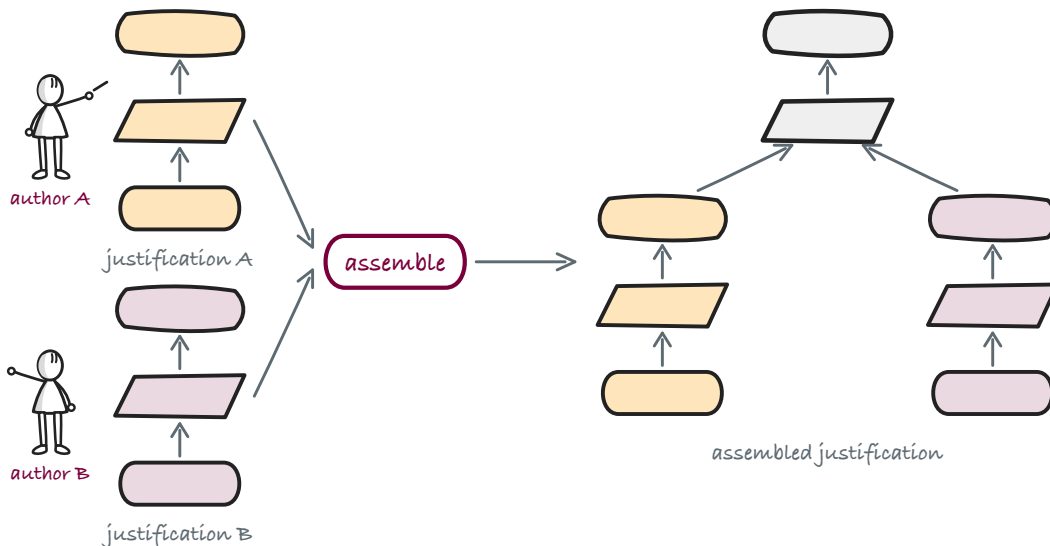
Link with Requirements



Templates can be partially implemented

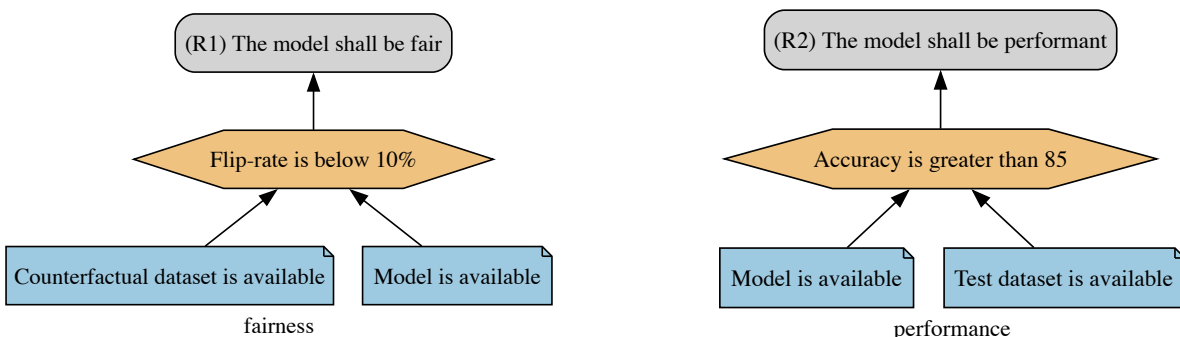


What if you don't know the requirements upfront?



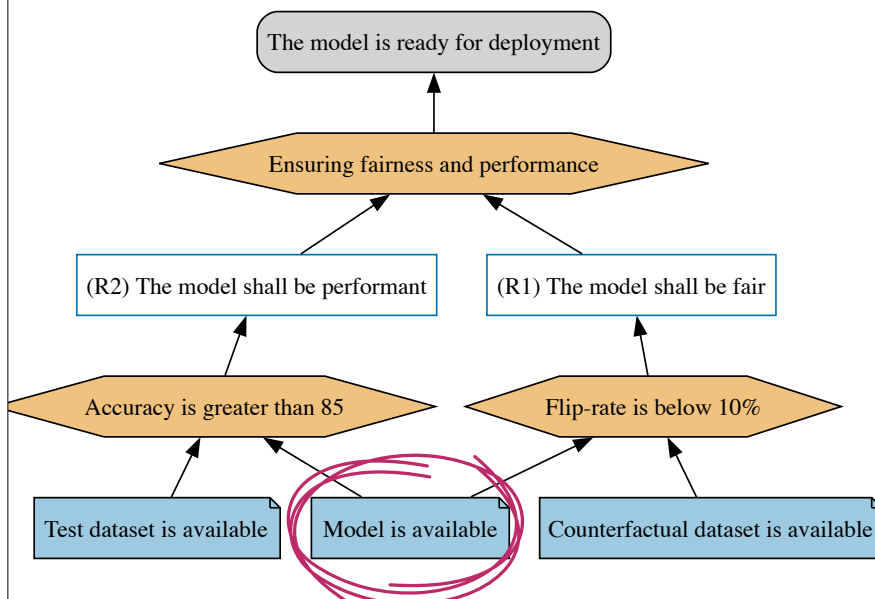
Lesson learned: Requirements Engineers love templates; ML Engineers love assemblies

Separation of concerns: Fairness & Perf.



```
justification deployable is assemble(performance, fairness) {
  conclusionLabel: "The model is ready for deployment"
  strategyLabel: "Ensuring fairness and performance"
}
```

Equivalent elements are unified



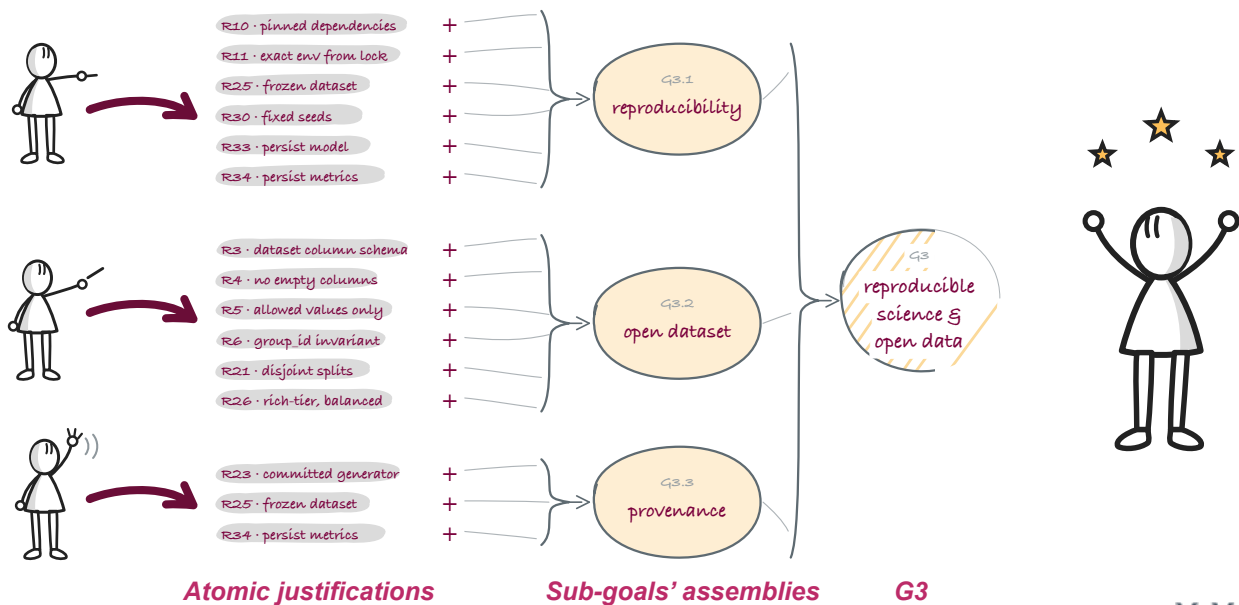
JPIPE Preview

Problems 0 Models 3 Symbols 17 Actions 38 Copy Report Text JSON

Filter...

fairness justification			
fair	conclusion		2:15
flip	strategy		3:15
counterfactual	evidence		4:15
model	evidence		5:15
performance justification			
performant	conclusion		13:15
accuracy	strategy		14:15
model	evidence		15:15
test	evidence		16:15
deployable justification			
assembleConclusion	conclusion	SYNTHESIZED	22:14
performance:performant	sub-conclusion		13:15
fairness:fair	sub-conclusion		2:15
performance:accuracy	strategy		14:15
fairness:flip	strategy		3:15
assembleStrategy	strategy	SYNTHESIZED	22:14
unified_0	evidence		15:15
performance:test	evidence		16:15
fairness:counterfactual	evidence		4:15
performance:model	→ unified_0	alias	
fairness:model	→ unified_0	alias	

Back to the emotion detection pipeline



Modelling with jPipe

Models can be organized in multiple files, as a catalogue

```
load "requirements/r{10,11,25,30,33,34}.jd"
load "requirements/r{3,4,5,6,21,26}.jd"
load "requirements/r23.jd"
```

```
justification reproducibility is assemble(r10, r11, r25, r30, r33, r34) {
  conclusionLabel: "The experiment is reproducible (G3.1)"
  strategyLabel: "Pinned, frozen, seeded, and persisted"
}

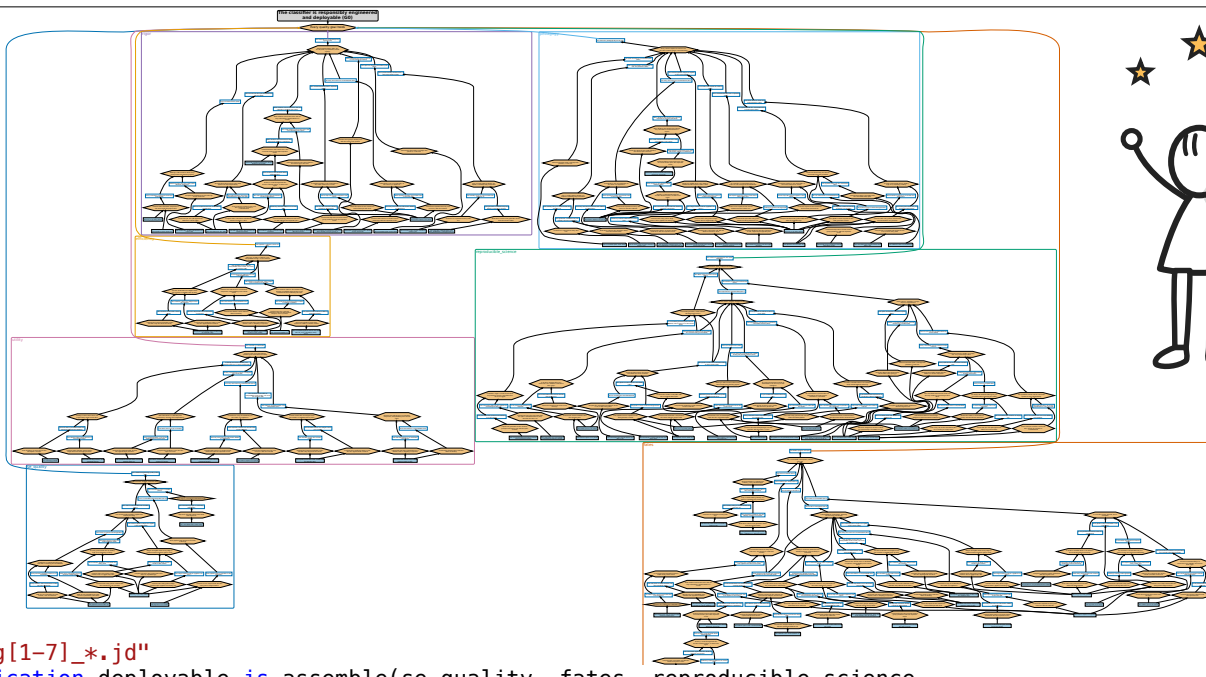
justification well_formed is assemble(r3, r4, r5) {
  conclusionLabel: "Every committed record conforms to the schema"
  strategyLabel: "Conformance fails by header, by empty cell, or by out-of-range value"
}

justification open_dataset is assemble(well_formed, r6, r21, r26) {
  conclusionLabel: "The dataset is open and well-formed (G3.2)"
  strategyLabel: "Fixed schema, complete, in-range, disjoint, and balanced"
}

justification provenance is assemble(r23, r25, r34) {
  conclusionLabel: "The data's origin is on the record (G3.3)"
  strategyLabel: "Committed generator, frozen artifact, self-describing metrics"
}

justification reproducible_science is assemble(reproducibility, open_dataset, provenance) {
  conclusionLabel: "The science is reproducible over open data (G3)"
  strategyLabel: "Reproducible, open, and provenanced"
}
```

> requirements
1 r1.jd
2 r2.jd
3 r3.jd
4 r4.jd
5 r5.jd
6 r6.jd
7 r7.jd
8 r8.jd
9 r9.jd
10 r10.jd
11 r11.jd
12 r13.jd
13 r14.jd
14 r15.jd
15 r16.jd
16 r17.jd
17 r18.jd
18 r19.jd
19 r20.jd
20 r21.jd
21 r22.jd
22 r23.jd
23 r24.jd
24 r25.jd
25 r26.jd
26 r27.jd
27 r28.jd
28 r29.jd
29 r30.jd
30 r31.jd
31 r32.jd
32 r33.jd
33 r34.jd
34 r35.jd
> steps
1 deployable.jd
2 g1_se_quality.jd
3 g2_fates.jd
4 g3_reproducible_sc.
5 g4_utility.jd
6 g5_pedagogy.jd
7 g6_efficiency.jd
8 g7_rigor.jd



```
load "g[1-7]*.jd"
justification deployable is assemble(se_quality, fates, reproducible_science,
                                     utility, pedagogy, efficiency, rigor) {
  conclusionLabel: "The classifier is responsibly engineered and deployable (G0)"
  strategyLabel: "Every quality goal holds"
}
```

Modelling a Justification

Time to practice on your own computer

3

Distribution Channels



(homebrew)

```
$ brew tap jpipe-mcscert/mcscert  
$ brew install jpipe
```

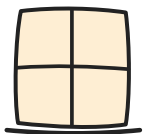


apt-get



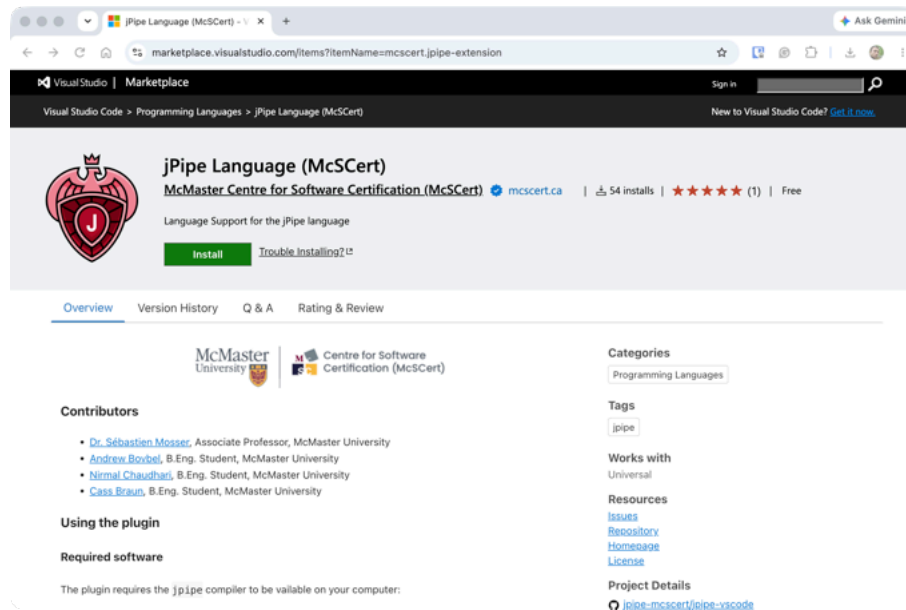
Ubuntu

```
$ sudo add-apt-repository ppa:mcscert/ppa  
$ sudo apt update  
$ sudo apt install jpipe
```

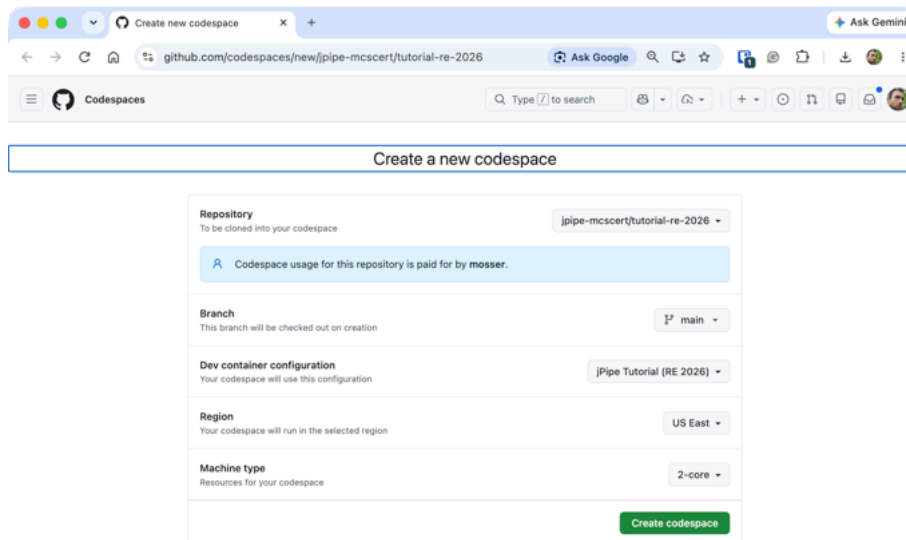


```
PS> scoop install git  
PS> scoop bucket add java  
PS> scoop bucket add mcscert https://...  
PS> scoop install mcscert/jpipe
```

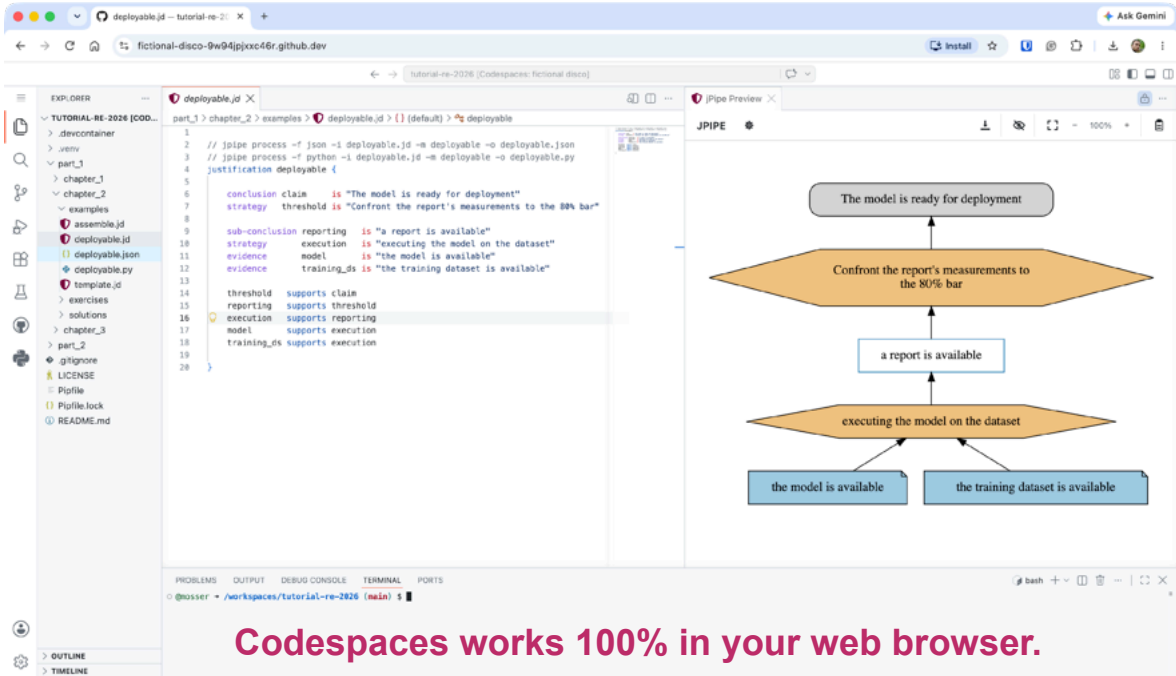
Install jPipe IDE from VSCode Marketplace



For this tutorial, use Codespace!



<https://codespaces.new/jpipe-mcscert/tutorial-models-2026>



Codespaces works 100% in your web browser.



BRIGHTER WORLD | © Sébastien Mosser — Dept. of Computing and Software (CAS) — McMaster Centre for Software Certification (McSCert) — Faculty of Engineering

71



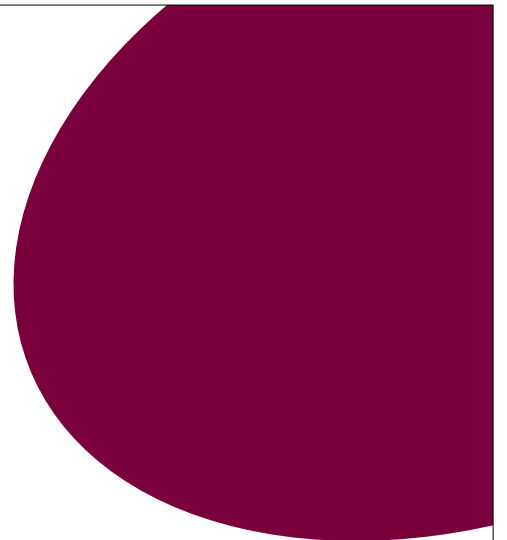
Time to write your first models!



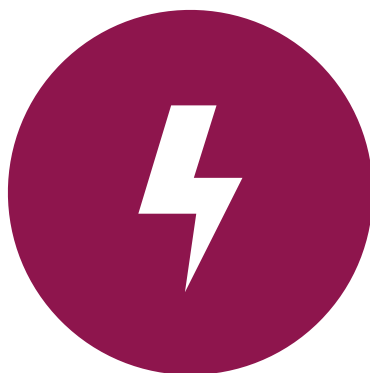
- **Open your Codespace** or local IDE
 - If not already: <https://bit.ly/code-models26>
 - Navigate to **part_1 >> chapter_3 >> README.md**
- **Tasks**
 1. **Write a justification** for graceful failure
 2. **Create a template** for “meeting a threshold”
 - *DRY: Don't Repeat Yourself*
 3. **Instantiate the template** twice.

Wrapping up

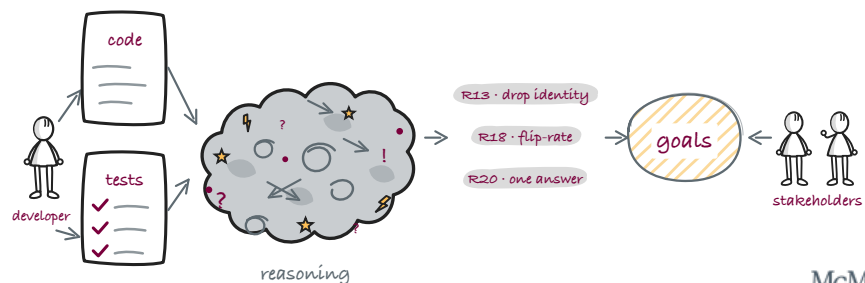
Key takeaways before getting some coffee



Summary



- **ML pipelines** come with **implicit requirements**
 - *Mistakes are made, decisions change, ...*
 - We need to **capture any drift**
- **jPipe** provides **Executable Justification Models**
 - Integrated into your **IDE** and **CI/CD** servers





ENGINEERING

Computing
& Software



McSCert



Dr. Sébastien Mosser
Associate Professor
mossers@mcmaster.ca



Calvin Thuan-Phong Khuu
PhD Student
khuu2@mcmaster.ca